



Java Performance Techniques. The Cost of Runtime Optimizations.

Ionuț Baloșin

Software Architect

ibalosin@luxoft.com

 [@ionutbalosin](https://twitter.com/ionutbalosin)

About Me

Software Architect @ www.luxoft.com

- 10+ years of software development experience

Technical Trainer @ www.luxoft-training.com

- JVA-028 - Java Performance and Tuning
- ARC-001 - Software Architecture Methodology
- ARC-008 - Designing High Performance Applications

Speaker, Blogger

Agenda

Just In Time Compiler Introduction

Uncommon Traps

Null Sanity Checks

Devirtualization

Loop Unrolling

String Deduplication

Biased Locking

Inlining & Concurrency Implications

Methods that do not JIT

Take Away

Better understanding about what happens under the hood inside JVM from JIT standpoint.

Learn about few JIT optimizations.

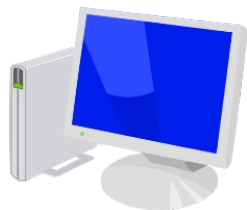
Cases when few JIT optimizations are prevented. How can be tackled such situations.

In a nutshell: it helps you to better tune your application from performance standpoint.

My Configuration

Hardware

- Intel i7-6700HQ Skylake
- 16GB DDR4 2133 MHz
- 1TB 5400 RPM+512GB PCIe SSD



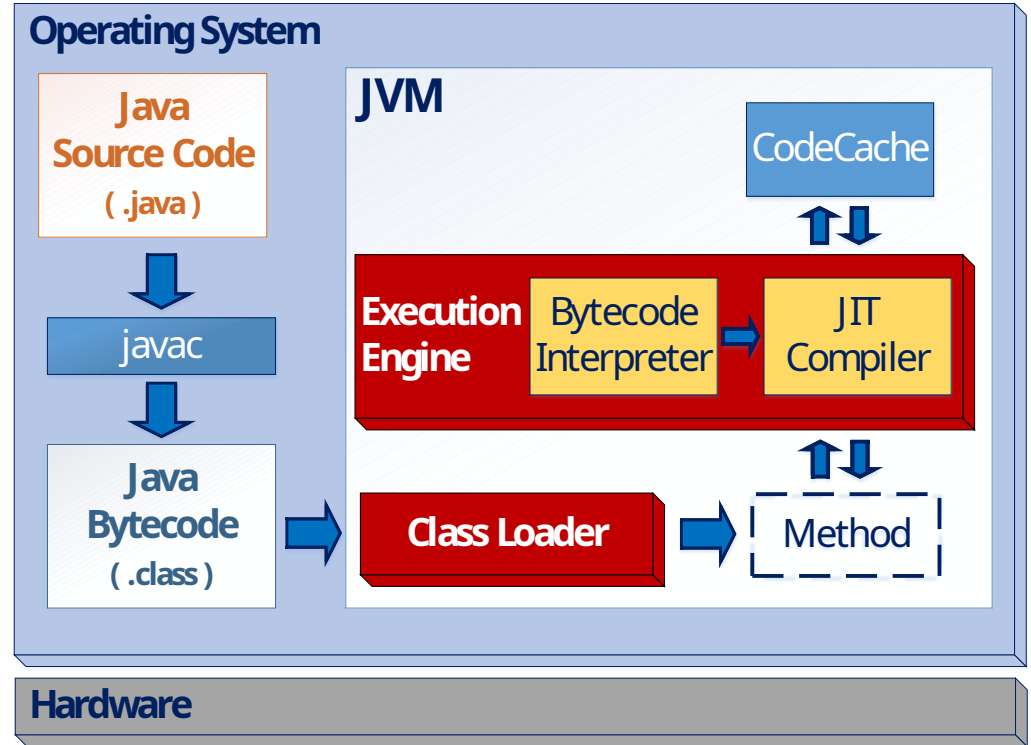
Software

- Ubuntu 16.04.2 & Win 10 Enterprise x64
- JDK 1.8.0_121 x64
- JITWatch
- Java Microbenchmark Harness
- HPjmeter



Just In Time Compiler Introduction

Just In Time Compiler - HotSpot



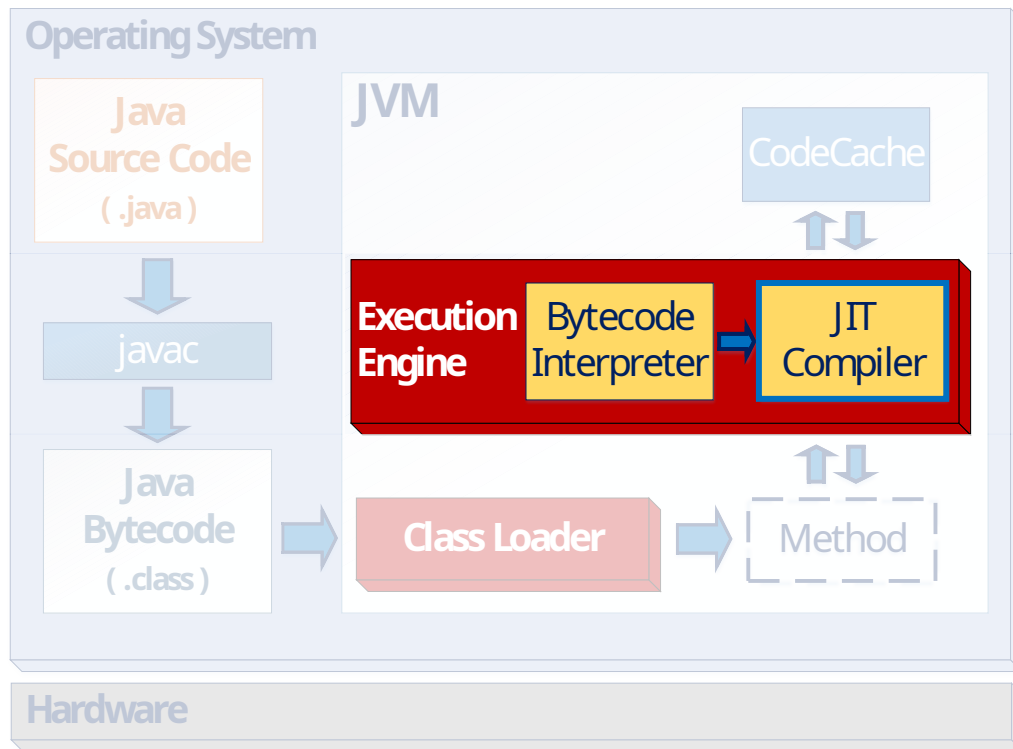
Just In Time Compiler - HotSpot

JIT Compiler

Client (C1)

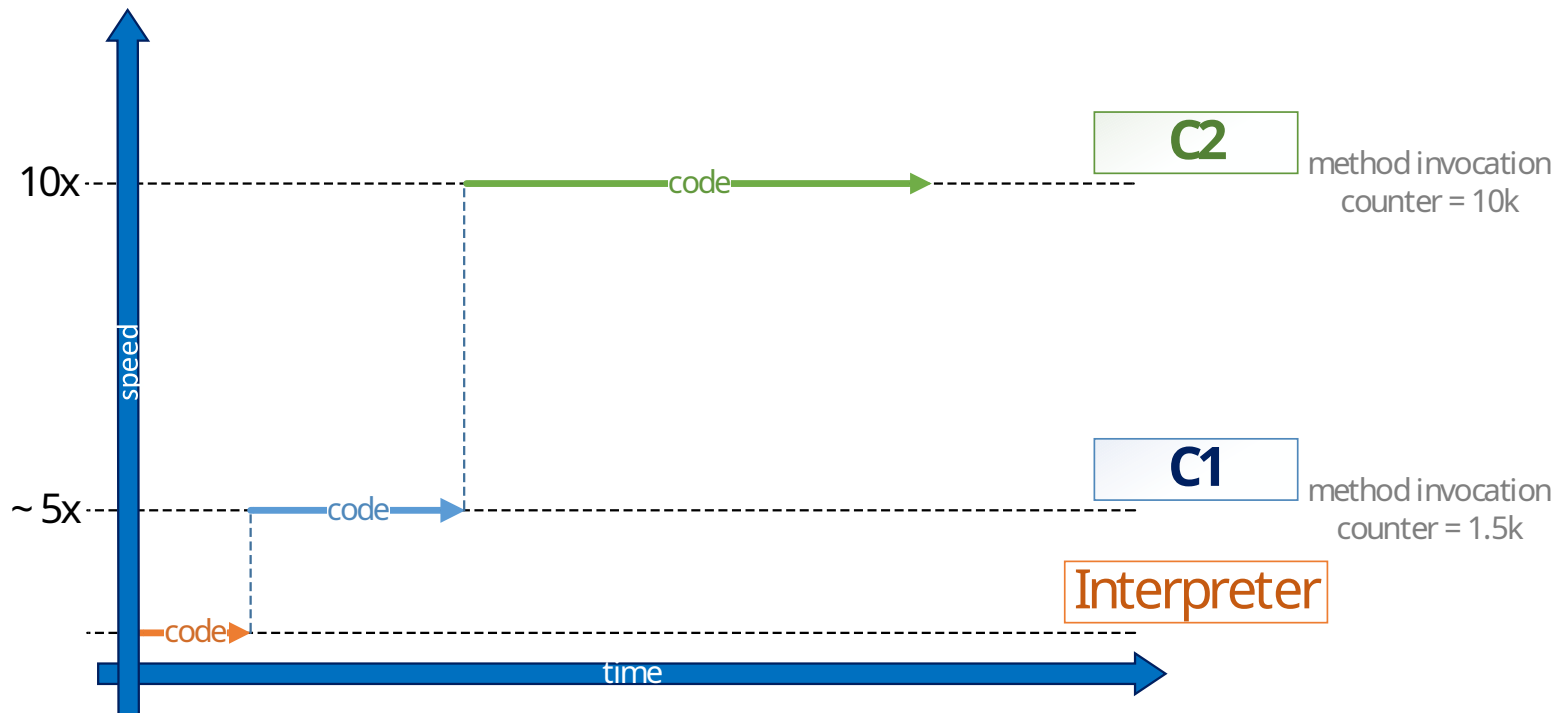
Server (C2)

Tiered Mode (C1 + C2)



Just In Time Compiler - HotSpot

Tiered Compilation Flows



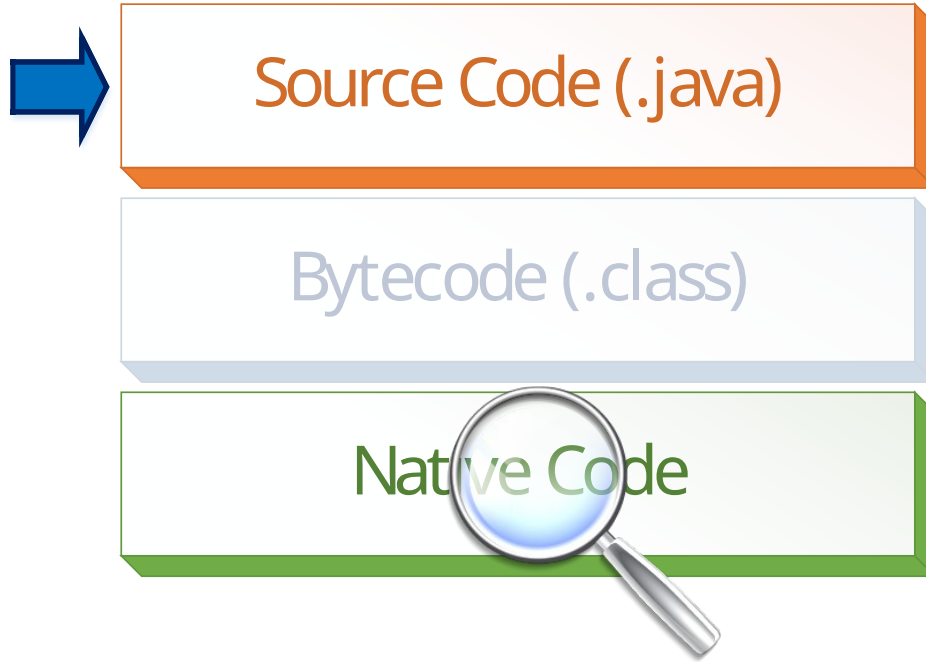
Experiments Rationale

Source Code (.java)

Bytecode (.class)

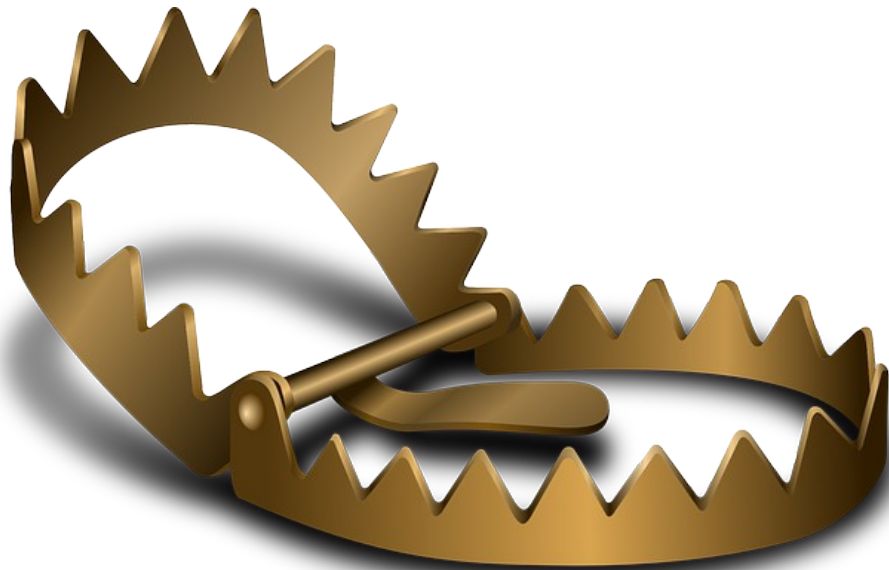
Native Code

Experiments Rationale



Uncommon Traps

Uncommon Traps



Uncommon trap is nothing more than a trampoline back to the Interpreter. The compiled code is de-optimized and is flagged as being unusable.

Uncommon Traps

➤ Code Sample

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(true);
}
```

```
private int hotMethod(boolean condition) {
    int counter;

    if (condition == true)
        counter = 1;
    else
        counter = 2;

    return counter;
}
```

Uncommon Traps

`$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation UncommonTraps.class`

```
# {method} {0x00000000104004f0} 'hotMethod'  
# parm0:    rdx          = boolean  
#          [sp+0x20]    (sp of caller)  
[Entry Point]  
...  
cmp edx,0x1    ; if (condition == true)  
jne L0000      ; *if_icmpne  
               ; - UncommonTraps::hotMethod@2 (line 52)  
mov eax,0x1    ; returns 1  
...
```

Uncommon Traps

\$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation UncommonTraps.class

```
# {method} {0x00000000104004f0} 'hotMethod'
# parm0:    rdx          = boolean
#           [sp+0x20]    (sp of caller)
[Entry Point]
...
cmp  edx,0x1          ; if (condition == true)
jne  L0000            ; *if_icmpne
                        ; - UncommonTraps::hotMethod@2 (line 52)
mov  eax,0x1          ; returns 1
...
```

```
L0000: mov  ebp,edx
      mov  edx,0xffffffff65
      xchg ax,ax
      call 0x00000000034157a0 ; OopMap{off=48}
                        ; *if_icmpne
                        ; - UncommonTraps::hotMethod@2 (line 52)
                        ; {runtime_call}
```

uncommon trap

JIT optimized (*condition == true*) branch. To handle (*condition == false*) an uncommon trap was added

Uncommon Traps

```
int sum = 0;  
for (int i = 0; i < 20_000; i++) {  
    sum += hotMethod(true);  
}
```

optimized & compiled

Uncommon Traps

```
int sum = 0;  
for (int i = 0; i < 20_000; i++) {  
    sum += hotMethod(true);  
}
```

optimized & compiled

Let's check now how long takes one `hotMethod(true)` call

```
hotMethod(true);    // ~ 144 ± 33 ns
```

But ... what if calling `hotMethod(false)` immediately after!?

```
hotMethod(false);   // ~ 14,000 ± 50 ns  Ouch ☹ !
```

Uncommon Traps

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(true);
}
```

optimized & compiled

Let's check now how long takes one `hotMethod(true)` call

```
hotMethod(true);    // ~ 144 ± 33 ns
```

But ... what if calling `hotMethod(false)` immediately after!?

```
hotMethod(false);   // ~ 14,000 ± 50 ns  Ouch ☹ !
```

Why so big difference between these two calls !?

Uncommon Traps

```
$> java -XX:+PrintCompilation -XX:-TieredCompilation UncommonTraps.class
```

```
// hotMethod(true) called 20_000 times
146   12   com.luxoft.jpt.conference.UncommonTraps::hotMethod (14 bytes)
146   13 %   com.luxoft.jpt.conference.UncommonTraps::main @ 4 (231 bytes)

// hotMethod(false) called 1 time
149   12   com.luxoft.jpt.conference.UncommonTraps::hotMethod (14 bytes) made not entrant
```

compiled bytecode size

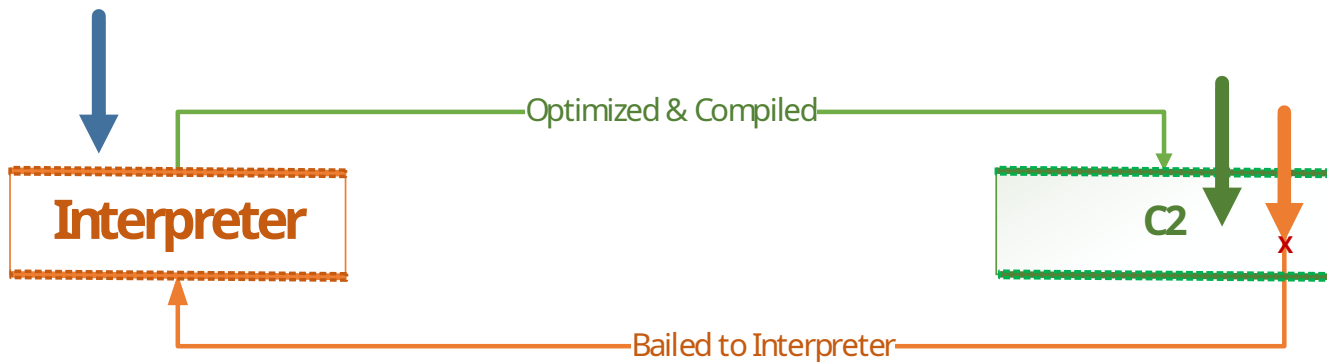
bytecode index which triggered OSR (osr_bci)

On-Stack-Replacement compilation_id

timestamp

** **made not entrant** - no future callers to the broken code*

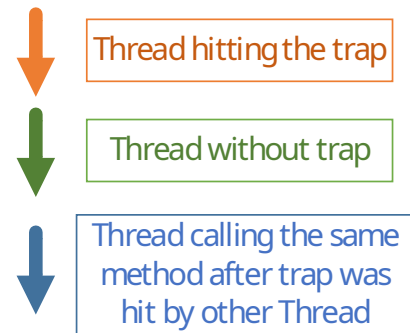
Uncommon Traps



*Current thread which hits the trap bails back to **Interpreter**.*

Another thread running the same method do not bail to Interpreter as long as it does not hit the trap.

*After bailing to **Interpreter** all new method calls by other threads will use this method version.*



Uncommon Traps

- Let's try to optimize `hotMethod(boolean)` for both `true` and `false` conditional branches

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(true);
}
// ...
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(false);
}
```

Uncommon Traps

- Let's try to optimize `hotMethod(boolean)` for both `true` and `false` conditional branches

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(true);
}
// ...
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(false);
}
```

```
$> java -XX:+PrintCompilation -XX:-TieredCompilation UncommonTraps.class
```

```
// hotMethod(true) called 20_000 times
```

```
162 12    com.luxoft.jpt.conference.UncommonTraps::hotMethod (14 bytes)
```

```
162 13 %   com.luxoft.jpt.conference.UncommonTraps::main @ 4 (74 bytes)
```

```
// hotMethod(false) called 20_000 times
```

```
163 12    com.luxoft.jpt.conference.UncommonTraps::hotMethod (14 bytes) made not entrant
```

```
163 14    com.luxoft.jpt.conference.UncommonTraps::hotMethod (14 bytes)
```

```
163 15 %   com.luxoft.jpt.conference.UncommonTraps::main @ 28 (74 bytes)
```

Uncommon Traps

```
$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation UncommonTraps.class
```

```
# {method} {0x00000000fa906c0} 'hotMethod' '(Z)I' in 'UncommonTraps'  
# parm0:    rdx          = boolean  
#          [sp+0x20]    (sp of caller)  
[Entry Point]  
...  
mov r11d,0x2  
mov eax,0x1  
cmp edx,0x1      ; if (condition == true)  
cmovne eax,r11d  ; *iload_1  
                  ; - UncommonTraps::hotMethod@12 (line 59)  
...
```

```
hotMethod(..);    // ~170 ± 47 ns
```

Not any **uncommon trap**, JIT falls back to a classic way to handle branches!

Uncommon Traps

Sum Up

If a conditional statement was following only a single branch, an **uncommon trap** is a good idea to optimize this.

In case the other branch was called, JIT falls back to the classic way of handling conditional statements (e.g. `cmovne`).

Triggering **uncommon trap** is costly since they fall back in Interpreter.

Uncommon Traps

World of Uncommon Traps



A word cloud of uncommon traps. The words are arranged in a roughly circular shape, with 'null_check' and 'range_check' being the largest and most prominent. Other words include 'class_check', 'bimorphic', 'constraint', 'unloaded', 'uninitialized', 'array_check', and 'unhandled'.

uninitialized
constraint
unloaded
bimorphic
class_check
null_check
range_check
array_check
unhandled

Null Sanity Checks

Null Sanity Checks

➤ Code Sample

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(17);
}
```

```
private int hotMethod(Integer anInt) {
    int counter = 0;

    if (null != anInt)
        counter = anInt * 2;

    return counter;
}
```

Null Sanity Checks

\$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation NullSanityChecks.class

```
# {method} {0x00000000fffb0668} 'hotMethod' '(Ljava/lang/Integer;)I' in  
'NullSanityChecks'  
# parm0:    rdx:rdx    = 'java/lang/Integer'  
#          [sp+0x30]  (sp of caller)  
[Entry Point]  
...  
mov eax,DWORD PTR [rdx+0xc] ; implicit exception: dispatches to 0x0000000002f7f1dd  
shl eax,1 ;*imul  
          ; - NullSanityChecks::hotMethod@12 (line 33)  
...
```

implicit NULL check



Null Sanity Checks

\$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation NullSanityChecks.class

```
# {method} {0x00000000fffb0668} 'hotMethod' '(Ljava/lang/Integer;)I' in  
'NullSanityChecks'  
# parm0:    rdx:rdx    = 'java/lang/Integer'  
#          [sp+0x30]   (sp of caller)  
[Entry Point]  
...  
mov eax,DWORD PTR [rdx+0xc] ; implicit exception: dispatches to 0x0000000002f7f1dd  
shl eax,1 ;*imul  
; - NullSanityChecks::hotMethod@12 (line 33)  
...
```

implicit NULL check

```
0x0000000002f7f1dd: mov QWORD PTR [rsp],rdx  
mov edx,0xffffffff65  
nop  
call 0x0000000002f557a0 ; OopMap{[0]=Oop off=44}  
; *if_acmpeq  
; - NullSanityChecks::testNullChecks@4 (line 32)  
; {runtime_call}
```

NULL handler
- uncommon trap -

JIT leverages on default OS signals (e.g. SEG_FAULT) to throw NPEs

Null Sanity Checks

- Let's try to optimize `hotMethod(Integer)` for both `NULL` and `!NULL` branches

```
int sum = 0;
for (int i = 0; i < 20_000; i++)
    sum += hotMethod(17);

sum += hotMethod(null);

for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(17);
}
```

Null Sanity Checks

```
# {method} {0x00000000fa60658} 'hotMethod' '(Ljava/lang/Integer;)I' in 'NullSanityChecks'
# parm0:    rdx:rdx    = 'java/lang/Integer'
#          [sp+0x20]   (sp of caller)
[Entry Point]
...
    test rdx,rdx      ← explicit NULL check
    je L0001          ;*if_acmpeq
                        ; - NullSanityChecks::hotMethod@4 (line 31)
    mov eax,DWORD PTR [rdx+0xc]
    shl eax,1         ;*iload_1
                        ; - NullSanityChecks::hotMethod@14 (line 34)
L0000: add rsp,0x10
    pop rbp
...
    ret
L0001: xor eax,eax
    jmp L0000
...
```

Not any **uncommon trap**, JIT falls back to a classic way to handle **NULL**!

Null Sanity Checks

Sum Up

NULL checks relying on signals (e.g. `SEG_FAULT`) are natively provided by hardware and handled by OS.

In case **NULL** branch is triggered JIT falls back to the classic way of handling **NULL** conditional statements (which is also costly).

Do not let **NULL** happen often in your application because it slows it down.

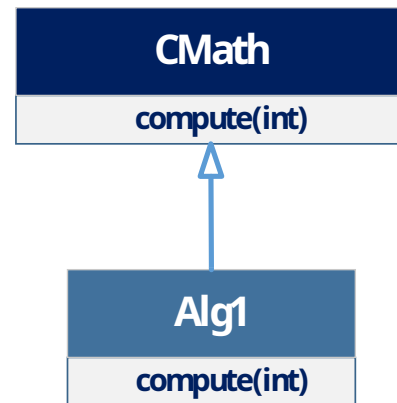
Devirtualization

Devirtualization

➤ Code Sample

```
double sum = 0;  
CMath alg1 = new Alg1();  
for (int i = 0; i < 20_000; i++) {  
    sum += hotMethod(alg1, i);  
}
```

```
private double hotMethod(CMath math, int i) {  
    return math.compute(i);    // return (double) (const * i)  
}
```



Devirtualization

```
$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation CHAVirtualCall.class
```

```
# {method} {0x00000000ffa05a0} 'hotMethod' '(LCMath;I)D' in 'CHAVirtualCall'
# parm0:    rdx:rdx    = 'CMath'
# parm1:    r8         = int
#           [sp+0x20]  (sp of caller)
[Entry Point]
...
vcvtsi2sd xmm0,xmm0,r8d                ; convert r8:int into xmm0:double precission
vmulsd xmm0,xmm0,QWORD PTR [rip+0xfffffffffffffc2] # 0x000000000302eec0
                                           ; multiply xmm0 by const
                                           ; *dmul
                                           ; - Alg1::compute@5 (line 58)
                                           ; - CHAVirtualCall::hotMethod@2 (line 37)
                                           ; {section_word}
...

```

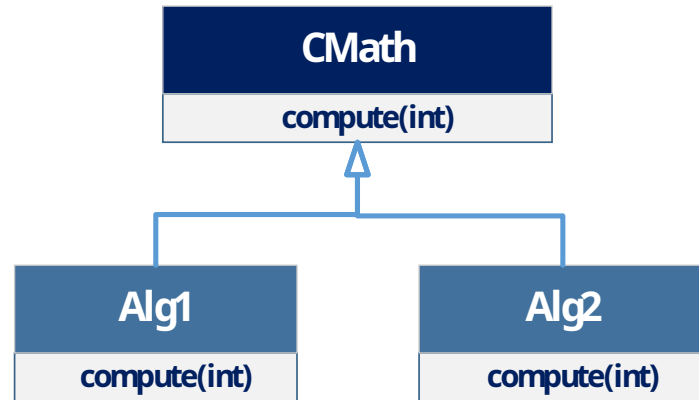
Monomorphic call (single target) is inlined without any **uncommon trap**.

Devirtualization

➤ Code Sample

```
double sum = 0;
CMath alg1 = new Alg1();
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(alg1, i);
}

CMath alg2 = new Alg2();
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(alg2, i);
}
```



Devirtualization

```
$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation CHAVirtualCall.class
```

```
# {method} {0x00000000fde0610} 'hotMethod' '(LCMath;I)D' in 'CHAVirtualCall'
# parm0:    rdx:rdx    = 'CMath'
# parm1:    r8         = int
#           [sp+0x30]   (sp of caller)
[Entry Point]
...
    vcvtsi2sd xmm0,xmm0,r8d    ;*i2d    ; - Alg2::compute@1 (line 69)
    cmp r10d,0xf800c082        ; {metadata('Alg1')}
    je L0000
    cmp r10d,0xf800c0c1        ; {metadata('Alg2')}
    jne L0002
    vmulsd xmm0,xmm0,QWORD PTR [rip+0xfffffffffffffb9] # 0x0000000002e2b2a8
                                                    ; - Alg2::compute@5 (line 69)
    jmp L0001                  ; method return
L0000: vmulsd xmm0,xmm0,QWORD PTR [rip+0xfffffffffffffa7] # 0x0000000002e2b2a0
                                                    ; - Alg1::compute@7 (line 58)
...
    L0002: call 0x0000000002df57a0 ; OopMap{rbp=Oop off=88}
                                           ; - CHAVirtualCall::hotMethod@2 (line 37)
...
```



Bimorphic call (two targets) is also inlined but **uncommon trap** is added.

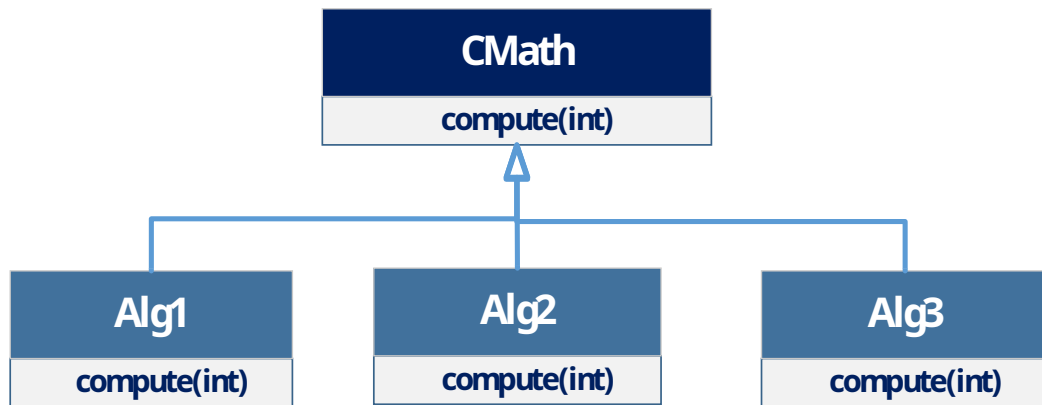
Devirtualization

➤ Code Sample

```
double sum = 0;
CMath alg1 = new Alg1();
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(alg1, i);
}

CMath alg2 = new Alg2();
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(alg2, i);
}

CMath alg3 = new Alg3();
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(alg3, i);
}
```



Devirtualization

```
$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation CHAVirtualCall.class
```

```
# {method} {0x0000000010260678} 'hotMethod' '(LCMath;I)D' in 'CHAVirtualCall'
# parm0:    rdx:rdx    = 'CMath'
# parm1:    r8         = int
#           [sp+0x20]  (sp of caller)
[Entry Point]
...
movabs rax,0xffffffffffffffff
call 0x00000000032063e0 ; OopMap{off=28}
                        ; *invokevirtual compute
                        ; - CHAVirtualCall::hotMethod@2 (line 37)
                        ; {virtual_call}
...
```

Megamorphic call is not anymore inlined, neither any **uncommon trap** is added.

Devirtualization

Sum Up

low



high

Monomorphic calls are inlined without any added **uncommon trap**.

Bimorphic calls are inlined but pay the cost of jump instructions and **uncommon trap**.

Megamorphic calls are handled as virtual calls.

The cost from monomorphic to megamorphic calls is ~ 3-7 ns.

Loop Unrolling

Loop Unrolling

➤ Code Sample

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod(array);
}
```

```
private int hotMethod(int[] array) {
    int sum = 0;

    for (int i = 0; i < array.length; i++) {
        sum += array[i];
    }

    return sum;
}
```

Loop Unrolling

\$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation LoopUnrolling.class

```
# {method} {0x00000000103f0538} 'hotMethod' '([I)I' in 'LoopUnrolling'
# parm0:    rdx:rdx    = '[I'
#           [sp+0x20]  (sp of caller)
[Entry Point]
...
    mov eax,DWORD PTR [rdx+0x10] ;*iaload - peels the first loop iteration
                                   ; - LoopUnrolling::hotMethod@13 (line 25)
...
    mov r9d,0x1
...
L0000: add eax,DWORD PTR [rdx+r9*4+0x10]
      movsxd r10,r9d
      add eax,DWORD PTR [rdx+r10*4+0x14]
      add eax,DWORD PTR [rdx+r10*4+0x18]
      add eax,DWORD PTR [rdx+r10*4+0x1c]
      add eax,DWORD PTR [rdx+r10*4+0x20]
      add eax,DWORD PTR [rdx+r10*4+0x24]
      add eax,DWORD PTR [rdx+r10*4+0x28]
      add eax,DWORD PTR [rdx+r10*4+0x2c] ;*iadd
                                   ; - LoopUnrolling::hotMethod@14 (line 25)
      add r9d,0x8 ;*iinc
                                   ; - LoopUnrolling::hotMethod@16 (line 24)
      cmp r9d,r8d
      jl L0000 ;*if_icmpge
                                   ; - LoopUnrolling::hotMethod@7 (line 24)
// handle the remaining array elements
...
```

Loop Unrolling

```
// peels the first loop iteration
sum = array[0]

i = 1
L0000:
    sum += array[i + 0]
    sum += array[i + 1]
    sum += array[i + 2]
    sum += array[i + 3]
    sum += array[i + 4]
    sum += array[i + 5]
    sum += array[i + 7]
    i += 8
cmp i,array.length
jl L0000

// handle the remaining array elements
```

Quizz

❑ Q: Would it be a good idea to do manual loop unrolling?

```
private static int hotMethod(int[] array) {  
    int sum = 0;  
  
    for (int i = 0; i < array.length; i += 4) {  
        sum += array[i];  
        sum += array[i + 1];  
        sum += array[i + 2];  
        sum += array[i + 3];  
    }  
  
    // handle the remaining array elements  
    return sum;  
}
```

Quizz

- ❑ A: Not really, let the JIT do the optimization by itself for your code ...

```
// peels the first loop iteration
```

```
sum = array[0]  
sum += array[1]  
sum += array[2]  
sum += array[3]
```

```
i = 4
```

```
L0000:
```

```
    sum += array[i]  
    sum += array[i + 1]  
    sum += array[i + 2]  
    sum += array[i + 3]  
    sum += array[i + 4]  
    sum += array[i + 5]  
    sum += array[i + 7]  
    i += 8
```

```
cmp i,array.length
```

```
jle L0000
```

```
// handle the remaining array elements
```

JIT recognizes it as an
unrolled loop and
rerolls it before
unrolling it again ... 😊

Loop Unrolling

Sum Up

Loop unrolling reduces the number of branches, hence removes some of the overhead of iterating through the entire loop.

Loop unrolling increases the degree of instruction level parallelism and might allow basic block level vectorization.

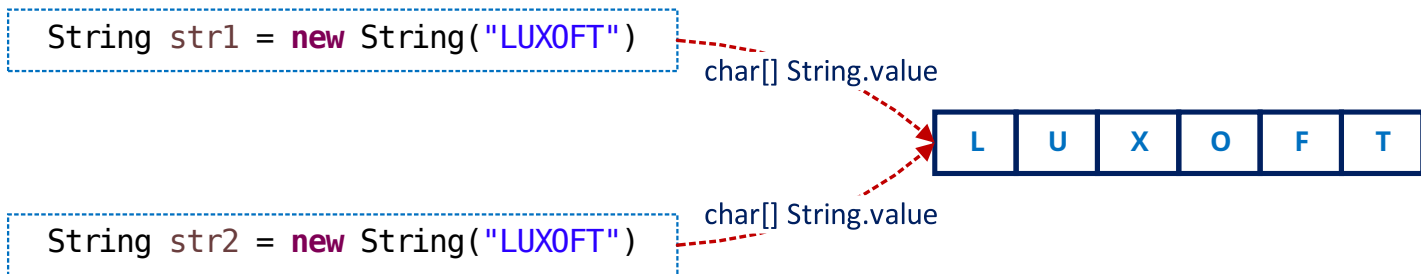
Avoid doing manual unrolling.

String Deduplication

String Deduplication

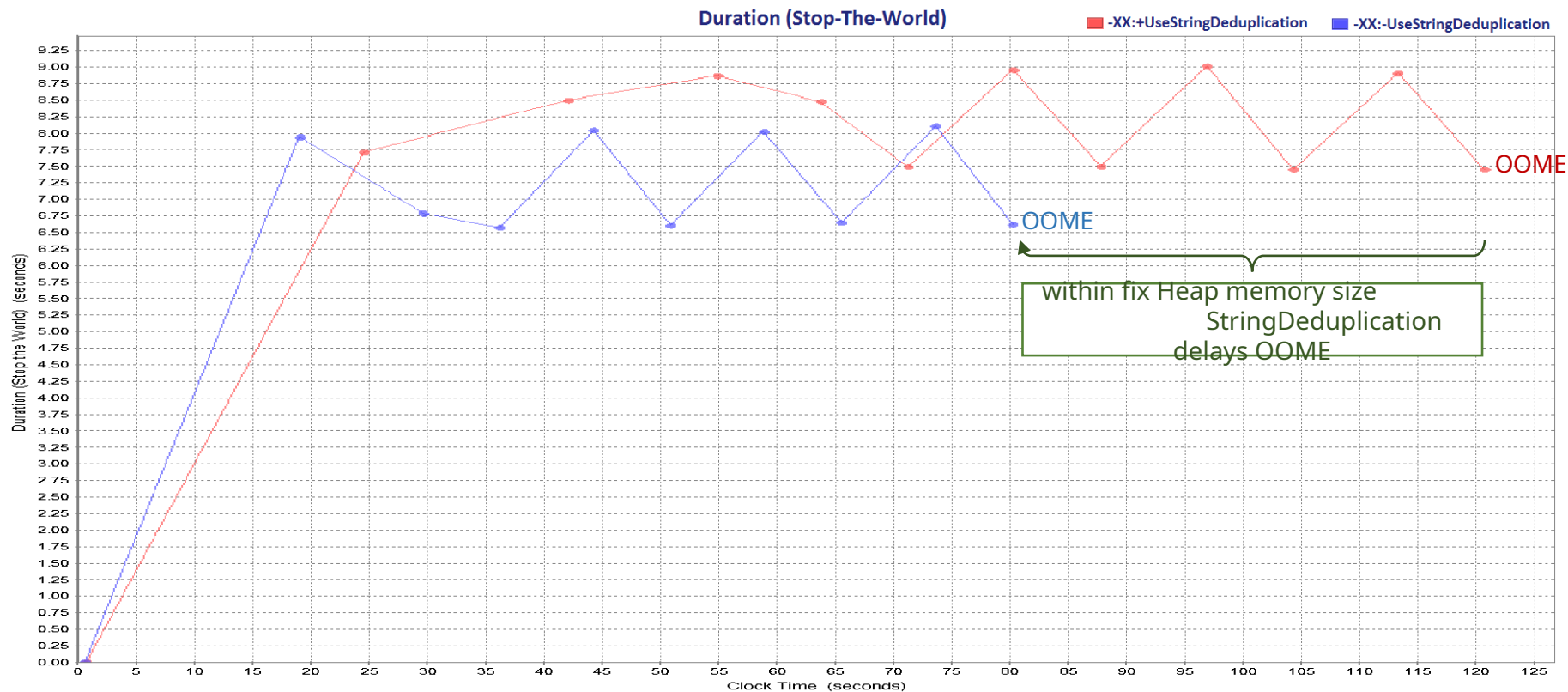
String Deduplication – Strings with equal `hashCode()` and the same underlying `char[]` become backed by just one underlying `char[]`

- underlying `char[]` no longer referenced are garbage collected
- processes strings which survived few GC cycles `-XX:StringDeduplicationAgeThreshold`
- available from Java 8 u20 with `-XX:+UseG1GC`



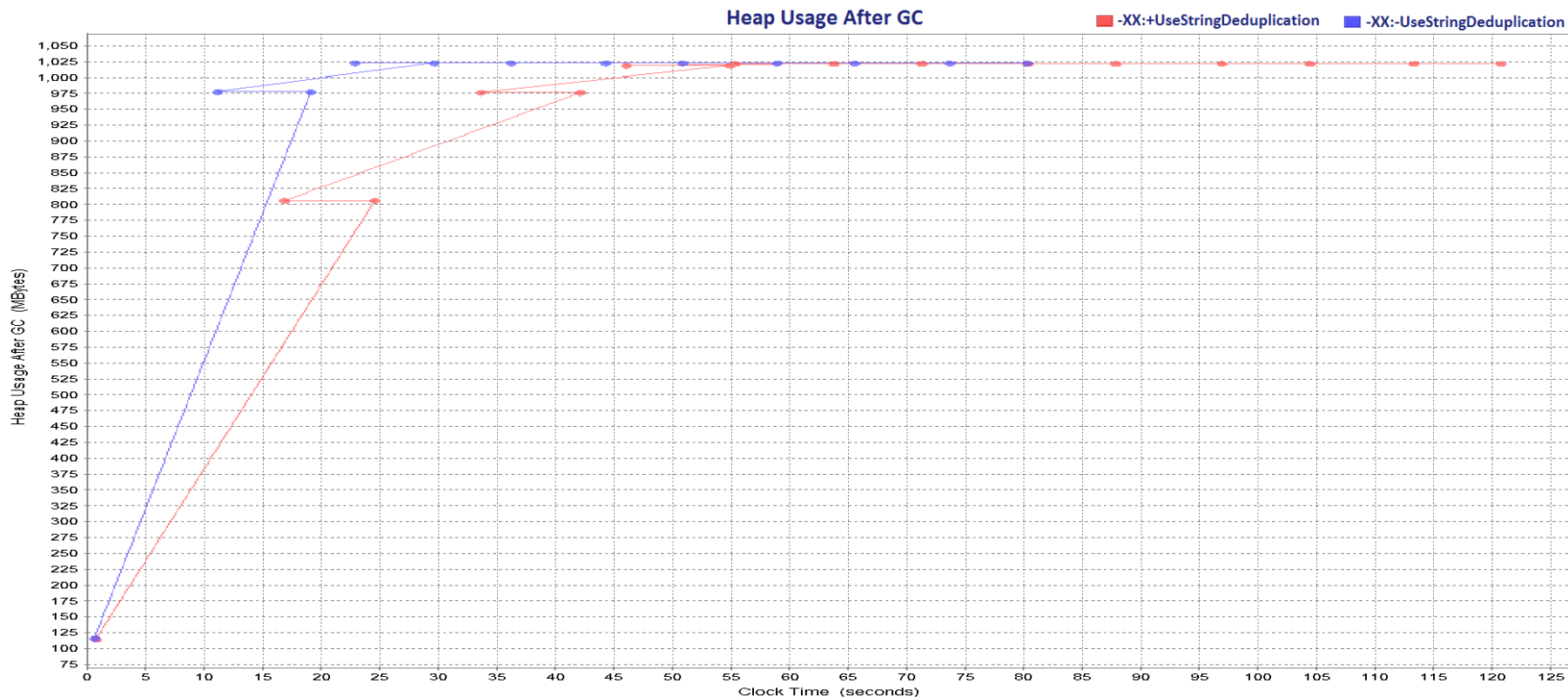
String Deduplication

Measure the impact of creating String objects using `-Xmx1024M -XX:+UseG1GC`



String Deduplication

Measure the impact of creating String objects using `-Xmx1024M -XX:+UseG1GC`



String Deduplication

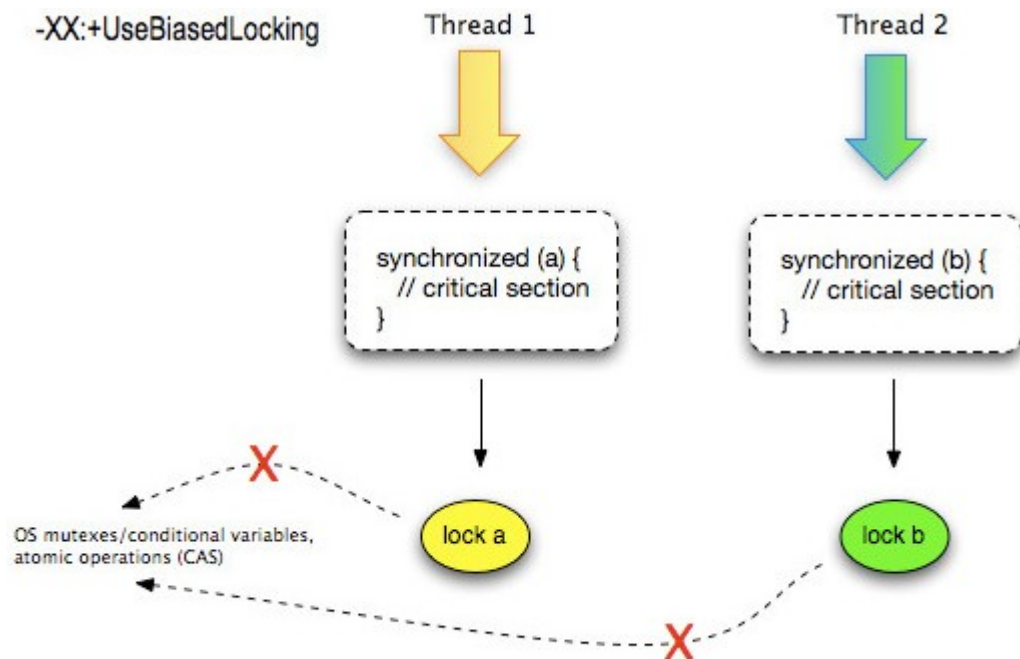
Sum Up

String Deduplication reduces the memory footprint and it is very useful in case of applications with a bounded amount of Heap memory

String Deduplication impacts performance due to time spent in matching the underlying `char[]` during GC cycles

Biased Locking

Biased Locking



Source: [<http://work.tinou.com/2009/06/lock-coarsening-biased-locking-escape-analysis-for-dummies.html>]

Biased Locking

➤ Code Sample

```
for (int i = 0; i < NO_OF_THREADS; i++) { // 50 threads
    Thread myThread = new Thread() {

        @Override
        public void run() {

            cyclicBarrier.await();

            synchronized (sharedInstance) {
                sharedInstance.counter++;
            }
        }
    };
    threads[i] = myThread;
}

// start threads
// join threads
```


Biased Locking

```
$> java -XX:BiasedLockingStartupDelay=0 -XX:+PrintSafepointStatistics -XX:+UnlockDiagnosticVMOptions  
-XX:+PrintGCApplicationStoppedTime LoopUnrolling.class
```

```
Total time for which application threads were stopped: 0.0012160 seconds, Stopping threads took: 0.0011872 seconds  
Total time for which application threads were stopped: 0.0000909 seconds, Stopping threads took: 0.0000249 seconds  
Total time for which application threads were stopped: 0.0000636 seconds, Stopping threads took: 0.0000273 seconds  
Total time for which application threads were stopped: 0.0000774 seconds, Stopping threads took: 0.0000419 seconds  
Total time for which application threads were stopped: 0.0000470 seconds, Stopping threads took: 0.0000249 seconds  
Total time for which application threads were stopped: 0.0000830 seconds, Stopping threads took: 0.0000320 seconds  
...
```

vmop	[threads: total initially_running wait_to_block]	[time: spin block sync cleanup vmop]	page_trap_count
0.087: EnableBiasedLocking	[10 0 1]	[0 1 1 0 0]	0
0.148: RevokeBias	[61 0 1]	[0 0 0 0 0]	0
...			
0.149: BulkRevokeBias	[43 0 2]	[0 0 0 0 0]	0
0.150: RevokeBias	[25 0 1]	[0 0 0 0 0]	0
...			
0.151: BulkRevokeBias	[15 0 2]	[0 0 0 0 0]	0
0.152: no vm operation	[10 0 1]	[0 0 0 0 316]	0

Polling page always armed

EnableBiasedLocking 1

RevokeBias 15

BulkRevokeBias 2

5 VM operations coalesced during safepoint

Maximum sync time 1 ms

Maximum vm operation time (except for Exit VM operation) 0 ms

Total time for which application threads were stopped: 0.0028275 seconds

Biased Locking

```
$> java -XX:-UseBiasedLocking -XX:+PrintSafepointStatistics -XX:+UnlockDiagnosticVMOptions -XX:  
+PrintGCApplicationStoppedTime LoopUnrolling.class
```

```
vmop      [threads: total initially_running wait_to_block]  [time: spin block sync cleanup vmop] page_trap_count  
0.315: no vm operation      [      10           0           1      ]      [      0      0      0      0      314      ]  0  
  
Polling page always armed  
  0 VM operations coalesced during safepoint  
Maximum sync time      0 ms  
Maximum vm operation time (except for Exit VM operation)      0 ms
```

Total time for which application threads were stopped: 0 seconds

Biased Locking

Sum Up

Biased locking helps a lot in case of un-contented locks, but if there is any contention it pays the cost of Stop-The-World pauses (e.g. RevokeBias).

In case your application deals with a lot of contention which might be known in advance, may be it is better to disable biased locking.

Inlining & Concurrency Implications

Inlining

It saves calls stack **POP/PUSH** overhead

It allows for powerful optimizations:

- if methods do not raise exceptions (consistent control flow) → JIT can move code around
- allows optimizations across method boundaries → optimize caller and callee as a big unit rather than in isolation
- if memory is not changed between method calls → saves number of memory loads

Inlining

➤ Code Sample

```
int sum = 0;
for (int i = 0; i < 20_000; i++) {
    sum += hotMethod();
}
```

```
private double hotMethod() {

    double h1 = rectangle.getHeight();    // return 42.0
    cantInlineMethod();
    double h2 = rectangle.getHeight();
    cantInlineMethod();
    double w1 = rectangle.getWidth();      // return 37.0
    cantInlineMethod();
    double w2 = rectangle.getWidth();

    return ((h1 * w1) + (h2 * w2));
}
```

Inlining

➤ Code Sample

```
private int cantInlineMethod() { return m0(); }
private int m0() { return m1(); }
private int m1() { return m2(); }
private int m2() { return m3(); }
private int m3() { return m4(); }
private int m4() { return m5(); }
private int m5() { return m6(); }
private int m6() { return m7(); }
private int m7() { return m8(); }
private int m8() { return m9(); }
private int m9() { return 0; }
```

```
$> java -XX:+UnlockDiagnosticVMOptions -XX:+PrintFlagsFinal -version | grep InlineLevel
    intx MaxInlineLevel           = 9           {product}
```

Inlining

\$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:-TieredCompilation LoopUnrolling.class

```
# {method} {0x00000000f650748} 'hotMethod' '()D' in 'SuccessiveMemoryLoads'
#          [sp+0x40] (sp of caller)
[Entry Point]
...
vmovsd xmm0,QWORD PTR [r12+r11*8+0x10] ;*getfield height
...
call 0x0000000025f6620 ; OopMap{rbp=Oop off=44}
                        ; - SuccessiveMemoryLoads::cantInlineMethod@0 (line 32)
...
vmovsd xmm0,QWORD PTR [r12+r10*8+0x10] ;*getfield height
...
call 0x0000000025f6620 ; OopMap{rbp=Oop off=68}
                        ; - SuccessiveMemoryLoads::cantInlineMethod@0 (line 32)
...
vmovsd xmm0,QWORD PTR [r12+r11*8+0x18] ;*getfield width
...
call 0x0000000025f6620 ; OopMap{rbp=Oop off=92}
                        ; - SuccessiveMemoryLoads::cantInlineMethod@0 (line 32)
...
vmovsd xmm0,QWORD PTR [r12+r10*8+0x18] ;*getfield width
...
```


Inlining

JIT assumes that memory holding *'height'* value could be changed by other threads

```
call rectangle.getHeight()  
call cantInlineMethod()  
call rectangle.getHeight()
```

... and forces a new memory load for *'height'* value

Inlining

`$> java -XX:+PrintAssembly -XX:PrintAssemblyOptions=intel -XX:MaxInlineLevel=10 -XX:-TieredCompilation LoopUnrolling.class`

```
# {method} {0x00000000f8a0748} 'hotMethod' '()D' in 'SuccessiveMemoryLoads'
#           [sp+0x20] (sp of caller)
[Entry Point]
...
// xmm0 = height x width
vmovsd xmm0,QWORD PTR [r12+r11*8+0x18] ; implicit exception dispatches to 0x00000000282a9f8
vmulsd xmm0,xmm0,QWORD PTR [r12+r11*8+0x10] ; *dmul
                                           ; - SuccessiveMemoryLoads::hotMethod@45 (line 36)
// xmm0 = xmm0 x xmm0
vaddsd xmm0,xmm0,xmm0 ; *dadd
                                           ; - SuccessiveMemoryLoads::hotMethod@50 (line 36)
...
```

Because `cantInlineMethod()` was now inlined, it reduced the memory loads in case of *height* and *width*.

Inlining

Sum Up

Inlining open doors towards other JIT optimizations!

-XX:MaxInlineSize	Default is 35 bytes
-XX:MinInliningThreshold	Default is 250
-XX:FreqInlineSize	Default is 325 bytes
-XX:MaxInlineLevel	Default is 9

Inlining

 AdoptOpenJDK / [jitwatch](#)

 Code

 Issues 17

 Pull requests 1

 Projects 0

 Wiki

 Pulse

 Graphs

JarScan

chriswhocodes edited this page on Nov 5, 2014 · 1 revision

What is the JarScan tool?

JarScan is a program included in the JITWatch source tree that statically analyses a jar file and counts the bytes in each method's bytecode.

It produces a CSV format report of the class, method signature, and bytecode size for each method that is above the 325 byte threshold for inlining hot methods (-XX:FreqInlineSize=n).

This report can be used to find methods that the HotSpot JIT compiler considers "hot" but were too big to be inlined into their caller. This information can be used to refactor hot methods into smaller sub-units of code that could potentially be inlined for greater performance.

The JarScan tool is launched via the included shell script jarScan.sh

Methods that do not JIT

Huge Methods

➤ Code Sample

```
private double hotMethod(int i) {  
    double d = 0;  
  
    d += 0x42; // Line 4  
    d += 0x42; // Line 5  
    d += 0x42; // Line 6  
    ...  
    d += 0x42; // Line 1335  
    d += 0x42; // Line 1336  
  
    return d + i;  
}
```

Method size is 8005 bytes



Huge Methods

```
$> java -XX:+PrintCompilation -XX:-TieredCompilation HugeMethods.class
```

```
293    12 %    com.luxoft.jpt.conference.HugeMethods::main @ 4 (50 bytes)
352    12 %    com.luxoft.jpt.conference.HugeMethods::main @ -2 (50 bytes)    made not entrant
```

TriView - Source, Bytecode, Assembly Viewer - JITWatch

Class: HugeMethods

Member: hotMethod(int)

Bytecode size: 3005 Native size: n/a Compile time: n/a

Source

```
1 public class HugeMethods {
2
3     //~ -----
4     //~ Methods
5     //~ -----
6
7     public static void main(String[] args) {
8
9         double sum = 0;
10        for (int i = 0; i < 20_000; i++) {
11            sum += hotMethod(i);
12        }
13
14        System.out.println("Sum = " + sum);
15    }
16
17    private static double hotMethod(int i) {
18        double d = 0;
19
20        d += 0x42;
21        d += 0x42;
22        d += 0x42;
23        d += 0x42;
```

Bytecode (double click for JVM spec)

```
1: dstore_1
2: dload_1
3: ldc2_w    #11 // double 66.0d
6: dadd
7: dstore_1
8: dload_1
9: ldc2_w    #11 // double 66.0d
12: dadd
13: dstore_1
14: dload_1
15: ldc2_w    #11 // double 66.0d
18: dadd
19: dstore_1
20: dload_1
21: ldc2_w    #11 // double 66.0d
24: dadd
25: dstore_1
26: dload_1
27: ldc2_w    #11 // double 66.0d
30: dadd
31: dstore_1
32: dload_1
```

Assembly

Labels

Not JIT-compiled

```
$> java -XX:+UnlockDiagnosticVMOptions -XX:+PrintFlagsFinal -version | grep DontCompile
bool DontCompileHugeMethods           = true      {product}
```

Huge Methods

```
$> java -XX:+PrintCompilation -XX:-TieredCompilation -XX:-DontCompileHugeMethods HugeMethods.class
```

```
234    12      com.luxoft.jpt.conference.HugeMethods::hotMethod (8005 bytes)
265    13 %    com.luxoft.jpt.conference.HugeMethods::main @ 4 (50 bytes)
```

JDK-6453531 : Severe performance regression on long methods for server VM

Details

Type: Bug	Submit Date: 2006-07-27
Status: Closed	Updated Date: 2011-02-23
Project Name: JDK	Resolved Date: 2006-07-27
Component: hotspot	OS: linux
Sub-Component: compiler	CPU: x86

Comments

EVALUATION

Compilation of very large methods (>8000 bytes) is not well-tested and expensive. The VM is tuned for compiling for the common programming paradigm, which includes generally small methods.

The user can work around the issue with the VM flag `-XX:-DontCompileHugeMethods`.

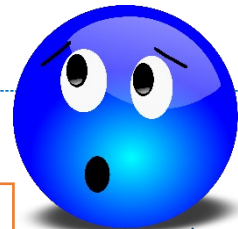
WORK AROUND

Use the VM flag `-XX:-DontCompileHugeMethods`

Too Many Arguments

➤ Code Sample

```
private double hotMethod(  
    //J+  
    double d00, double d01, double d02, double d03, double d04, double d05, double d06, double d07, double d08, double d09,  
    double d10, double d11, double d12, double d13, double d14, double d15, double d16, double d17, double d18, double d19,  
    double d20, double d21, double d22, double d23, double d24, double d25, double d26, double d27, double d28, double d29,  
    double d30, double d31, double d32, double d33, double d34, double d35, double d36, double d37, double d38, double d39,  
    double d40, double d41, double d42, double d43, double d44, double d45, double d46, double d47, double d48, double d49,  
    double d50, double d51, double d52, double d53, double d54, double d55, double d56, double d57, double d58, double d59,  
    double d60, double d61, double d62, double d63, double d64, double d65, double d66, double d67, double d68, double d69  
    //J-  
) {  
    return  
        //J-  
        d00 + d01 + d02 + d03 + d04 + d05 + d06 + d07 + d08 + d09 + d10 + d11 + d12 + d13 + d14 + d15 + d16 + d17 + d18 + d19 +  
        d20 + d21 + d22 + d23 + d24 + d25 + d26 + d27 + d28 + d29 + d30 + d31 + d32 + d33 + d34 + d35 + d36 + d37 + d38 + d39 +  
        d40 + d41 + d42 + d43 + d44 + d45 + d46 + d47 + d48 + d49 + d50 + d51 + d52 + d53 + d54 + d55 + d56 + d57 + d58 + d59 +  
        d60 + d61 + d62 + d63 + d64 + d65 + d66 + d67 + d68 + d69;  
        //J+  
}
```



Method has 70 doubles arguments

Too Many Arguments

```
$> java -XX:+PrintCompilation -XX:-TieredCompilation TooManyArgs.class
```

```
155    12      com.luxoft.jpt.conference.TooManyArgs::hotMethod (208 bytes)
156    12      com.luxoft.jpt.conference.TooManyArgs::hotMethod (208 bytes)
      COMPILE SKIPPED: unsupported incoming calling sequence (not retryable)
157    13 %    com.luxoft.jpt.conference.TooManyArgs::main @ 2 (226 bytes)
158    13 %    com.luxoft.jpt.conference.TooManyArgs::main @ 2 (226 bytes)
      COMPILE SKIPPED: unsupported calling sequence (not retryable)
```



JDK / JDK-5090493

COMPILE SKIPPED: 'unsupported calling sequence' due to method signature

Agile Board

Details

Type: Enhancement

Status: **OPEN**

Priority: **4** P4

Resolution: Unresolved

Affects Version/s: 6, 9, 10

Fix Version/s: None

Component/s: hotspot

Labels: **c2** **webbug**

Subcomponent: compiler

Understanding: Cause Known

In Closing ...

“It doesn't make a difference how beautiful your guess is.

It doesn't make a difference
how smart you are, who made the guess, or what his name
is.

If it disagrees with experiment, it's
wrong.”



Richard Feynman

The Feynman Lectures on Physics: The New Millennium Edition

In Closing ...

"It doesn't make a difference how beautiful your guess is.

It doesn't make a difference
how smart you are, who made the guess, or what his name
is.

If it disagrees with experiment, it's
wrong."



Richard Feynman

The Feynman Lectures on Physics: The New Millennium Edition

Always keep on experiencing new things !

A collection of various blue geometric shapes including triangles, squares, and circles, some containing icons like a gear and a lightbulb, scattered on the left side of the slide.

THANK YOU

Ionuț Baloșin

Software Architect

ibalosin@luxoft.com

 [@ionutbalosin](https://twitter.com/ionutbalosin)





Appendix

On Stack Replacement

◆

$$◆ \quad OSR Trigger = CompileThreshold * \frac{OnStackReplacePercentage - InterpreterProfilePercentage}{100}$$

OSR Trigger = 13,500 (-client)
OSR Trigger = 10,700 (-server)

-XX:CompileThreshold	1,500 (-client) / 10,000 (-server)
-XX:OnStackReplacePercentage	933 (-client) / 140 (-server)
-XX:InterpreterProfilePercentage	33

For tiered compilation, these decisions are based on different flags altogether, though the concepts are the same

String Deduplication

OpenJDK JEP 192: String Deduplication in G1

<i>Owner</i>	Per Liden
<i>Created</i>	2013/11/22 20:00
<i>Updated</i>	2017/02/15 07:02
<i>Type</i>	Feature
<i>Status</i>	Submitted
<i>Component</i>	hotspot / gc
<i>Scope</i>	Implementation
<i>Discussion</i>	hotspot dash gc dash dev at openjdk dot java dot net
<i>Effort</i>	M
<i>Duration</i>	L
<i>Priority</i>	4
<i>Reviewed by</i>	Bengt Rutisson, John Coomes, Jon Masamitsu
<i>Endorsed by</i>	Mikael Vidstedt
<i>Release</i>	8u20
<i>Issue</i>	8046182
<i>Relates to</i>	JEP 254: Compact Strings

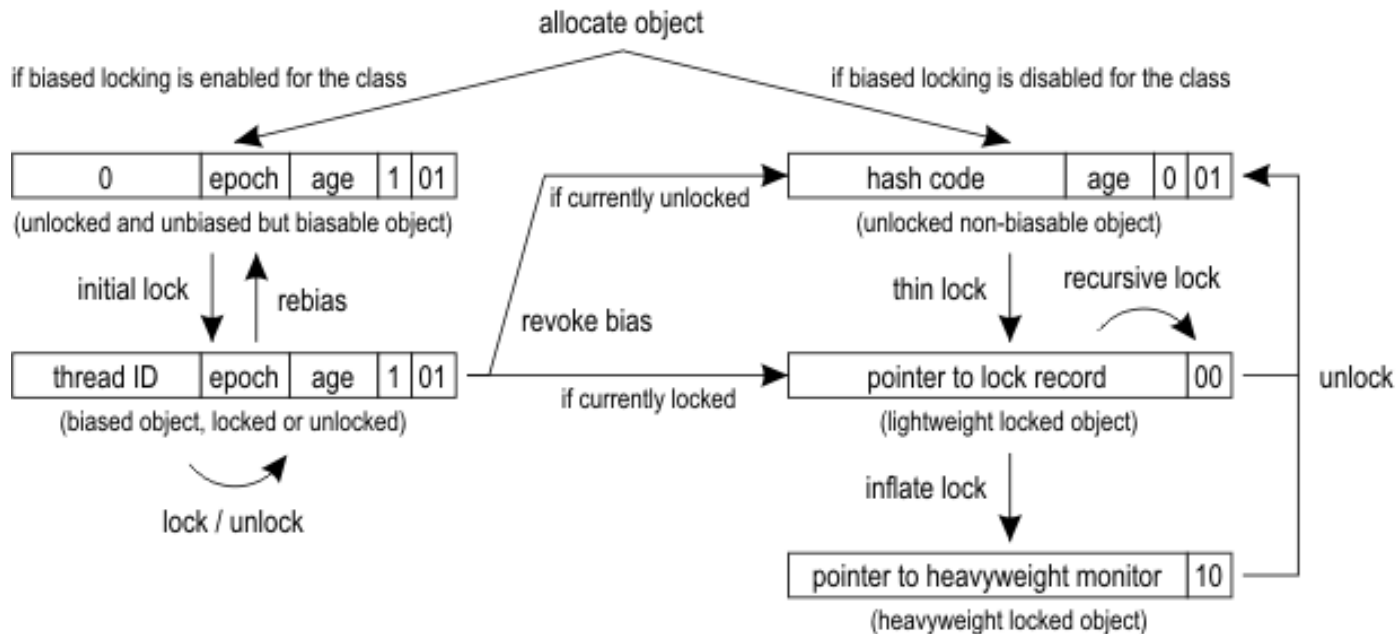
Summary

Reduce the Java heap live-data set by enhancing the G1 garbage collector so that duplicate instances of String are automatically and continuously deduplicated.

Non-Goals

It not a goal to implement this feature for garbage collectors other than G1.

Synchronization and Object Locking



Source: [<https://wiki.openjdk.java.net/display/HotSpot/Synchronization>]

Just In Time Compiler - HotSpot

Tiered Compilation Flows

